University OF Engineering and Technology, Lahore
Computer Engineering Department

Course Name: PF	**Course Code:** CMPE-112L
Assignment Type: lab	**Dated: 16 MARCH 2026**
Semester: 1st	**Session: Spring 2026**
Lab/Project/Assignment #: 3	**CLOs to be covered: CLO1**
Lab Title: Lambda functions	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Lab Goals/Objectives:

• **Indefinite Iteration:** Unlike a `for` loop (which usually runs a set number of times), a `while` loop is built for situations where the loop count depends on real-time events.

• **Waiting for an Event:** It keeps running until a certain state changes, like waiting for a file to finish downloading or waiting for a specific sensor reading.

• **User Input Validation:** It can repeatedly ask a user for input until they type something valid.

Equipment Required:

• Computer/Laptop
• Python 3.x
• VS Code / PyCharm / IDLE

THEORY:

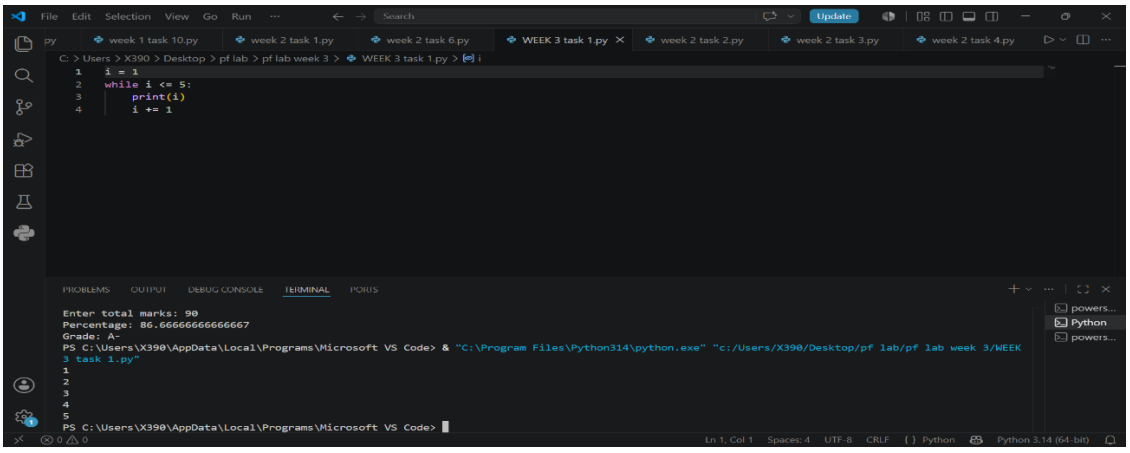
A `while` loop is a fundamental control flow structure designed for indefinite iteration, meaning it repeats a block of code for an unknown number of times based on a dynamic condition. It functions by continuously executing its internal statements as long as a specified boolean condition evaluates to true. Because this condition is tested before each iteration begins, the loop is considered a pre-test loop; if the condition is false from the start, the code inside will never run at all. To prevent the program from trapping itself in a system-freezing infinite loop, the code inside must eventually modify a variable that forces the condition to become false. Once that false state is reached, the loop immediately terminates, and the program skips to the next line of code. This behavior makes `while` loops uniquely ideal for event-driven programming tasks where you cannot predict the exact timing of a stop signal, such as continuously reading data streams, validating unpredictable user inputs, or driving core game loops

LAB TASKS:

TASK NO 1:

Write a Python program that utilizes a basic indefinite iteration loop to display a sequential range of positive integers starting from 1 up to a designated maximum ceiling (5)

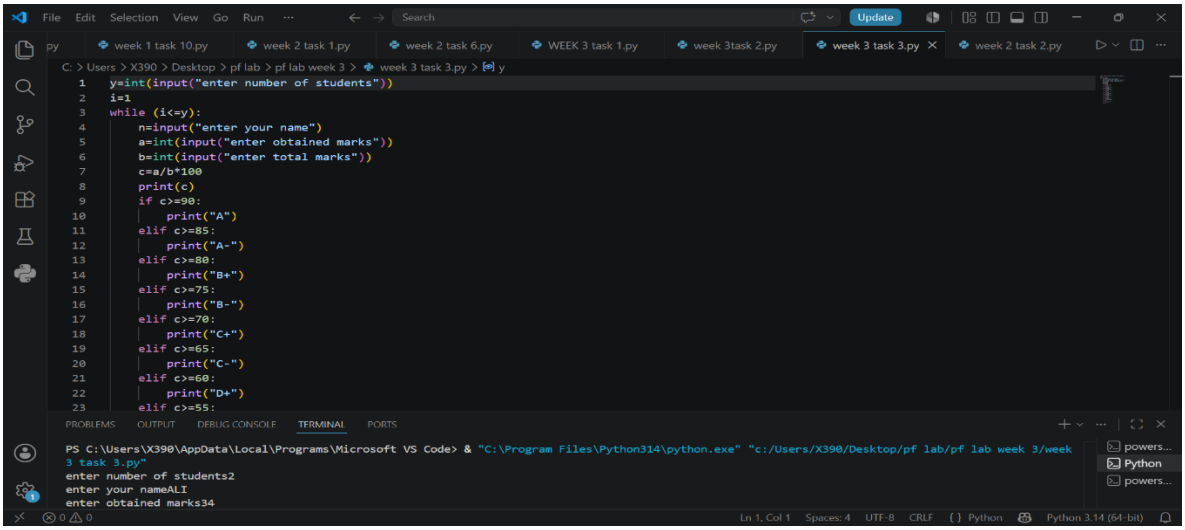
RESULT:



TASK NO 2:

Write a Python program that calculates the cumulative sum of all positive integers from 1 up to an inclusive upper limit of 10 using a loop control structure.

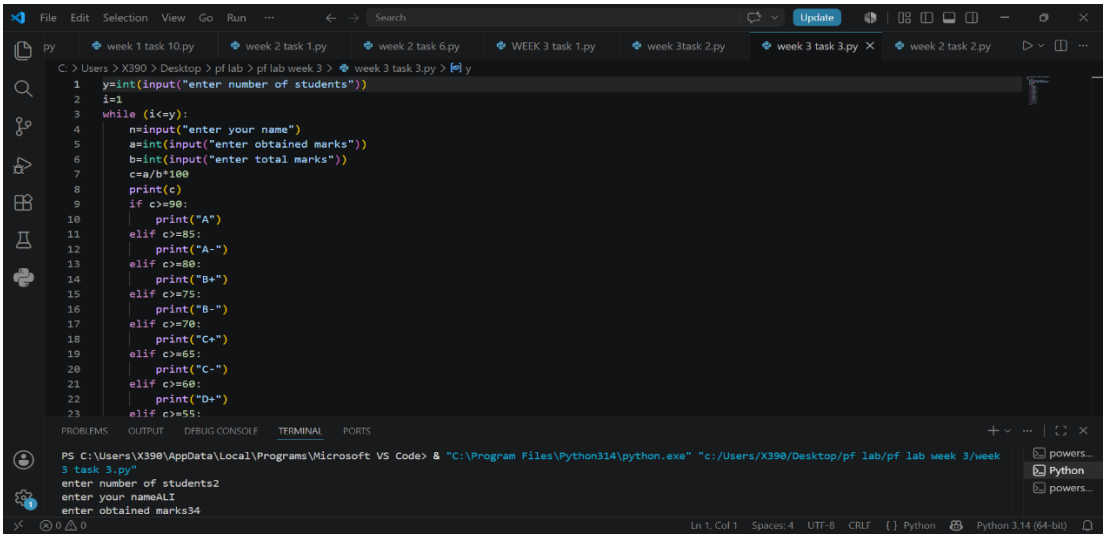
RESULT;



TASK NO 3:

Write a Python program using a `while` loop to process academic grades for a specific number of students. The program should accept student names, obtained marks, and total marks, calculate each student's final percentage, and map it to an extended alphabetical grading system

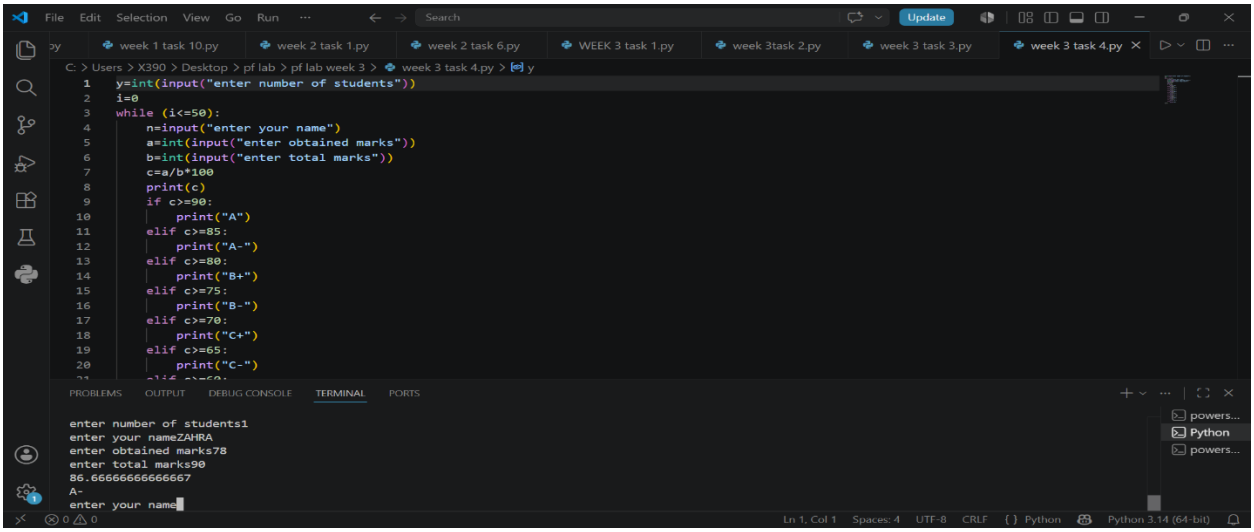
RESULT:



TASK NO 4:

Write a Python program using a `while` loop to process academic grades for a fixed capacity cohort of up to 50 students. The program should accept student names, obtained marks, and total marks, calculate each student's final percentage, and map it to an alphabetical grading system.

RESULT:



CONCLUSION:

This week's laboratory tasks successfully illustrate the foundational mechanics of indefinite iteration in Python using `while` loops. By progressing from basic sequential counting and mathematical accumulation to a multi-conditional student grading system, the exercises demonstrate how to systematically manage state variables and loop boundaries. However, the recurring bugs encountered—specifically the omission of counter increments and the presence of dead code paths—powerfully emphasize why loops require careful condition handling. They highlight how easily a script can become trapped in an infinite loop or crash due to an unindented statement if the state update is neglected. Ultimately, these tasks reinforce that while `while` loops offer excellent flexibility for open-ended execution, they demand strict defensive programming to ensure structural reliability and code stability